Algorithm Analysis - US27

Introduction

This document analyzes the **time complexity** of the algorithms implemented in **US27**, which focus on finding one of the shortest routes between two stations, passing through a chosen ordered list of stations. The algorithms involved are `dijkstra`, `findFastestPath`, and `buildStops`, which are used to compute the shortest paths, while taking into account specific conditions like whether the line is electrified.

The analysis will evaluate the **worst-case time complexity** using Big O notation, helping us understand the behavior of the algorithms as the number of stations (`n`) and the number of lines (`m`) in the network increase.

---

1. `dijkstra` Method

The `dijkstra` method computes the shortest path between two stations, taking into account the allowed connections and the weight of each railway line.

`dijkstra` Pseudocode Analysis

dijkstra(src, dst integer; allowed[n][n] boolean; weight[][] double, stations[])	Big-O Complexity
`dist[n]; prev[n]; used[n]`	**O(n)**
`for i = 1 to n`	**O(n)**
`dist[i] = 999999999; prev[i] = -1`	**O(n)**
`dist[src] = 0`	**O(1)**
for `it = 1 to n`	**O(n)**
--- `u = -1; best = 999999999`	**O(n)**
--- **for** `i = 0 to n`	**O(n²)**
---- `if(used[i] = 0 and dist[i] < best)`	**O(n²)**
------- `best = dist[i]; u = i`	**O(1)**
--- `if u == -1 or u == dst then break;`	**O(n)**
--- `used[u] = 1`	**O(n)**
--- **for** `v = 1 to n`	**O(n)**
------ `if (allowed[u][v] = 1)`	**O(n)**
------ `alt = dist[u] + weight[u][v]`	**O(n)**
------ `if (alt < dist[v]) { dist[v] = alt; prev[v] = u; }`	**O(n)**
`if (999999999 == dist[dst]) return null`	**O(1)**
`at = dst; i = 0`	**O(1)**
`while at != -1`	**O(n)**
--- `path[i] = stations[at]; at = prev[at]; i = i + 1`	**O(n)**
`return new DijkstraResult(path, dist[dst])`	**O(1)**

dijkstra(src, dst integer; allowed[n][n] boolean; weight[][] double, stations[])	Big-O Complexity
Total	**O(max(O(n²), ...))** **O(n²)**

Explanation:

1. **Initialization:** The algorithm first initializes arrays for distances, predecessors, and used vertices, each taking **O(n)** time.
2. **Main Loop:** The outer loop runs `n` times, and for each iteration, the algorithm searches for the minimum distance node, which takes **O(n²)** due to the inner loop checking all nodes.
3. **Path Construction:** The path is built by following the `prev` array from the destination to the source, which takes **O(n)** time.
4. **Total Complexity:** The overall time complexity is dominated by the **O(n²)** term, making the algorithm's complexity **O(n²)**.

---

2. `findFastestPath` Method

The `findFastestPath` method calculates the fastest route between two stations, considering a set of intermediate stations and whether the line is electrified.

`findFastestPath` Pseudocode Analysis

findFastestPath(stations[n], Lines[m], eletric boolean)	Big-O Complexity
for `i = 1 to n` `write(stations[i])`	**O(n)**
read `src`, read `dst`, read `k`	**O(1)**
`stops := buildStops(src, stations, src, dst, k)`	**O(n)**
`allowed[n][n], weight[n][n]`	**O(1)**
for `i = 1 to n`	**O(n)**
--- **for** `j = 1 to n`	**O(n²)**
------- `weight[i][j] = 999999999`	**O(n²)**
for `i = 1 to m`	**O(m)**
--- `if eletric == 1 and Lines[i].isEletrified == 1 then`	**O(m)**
------ `a = indexOf(Lines[i].A), b = indexOf(Lines[i].B)`	**O(n)**
------ `allowed[a][b] = allowed[b][a] = true; weight[a][b] = weight[b][a] = distance`	**O(n)**
`totalDist = 0; size = stops.getsize()`	**O(1)**
for `i = 1 to size`	**O(n)**
--- `iSrc = indexOf(stops[i]), iDst = indexOf(stops[i + 1])`	**O(n)**
--- `seg = dijkstra(iSrc, iDst, allowed, weight, stations)`	**O(n³)**
--- `if seg == null then print error & return`	**O(n)**
--- `totalDist += seg.distance()`	**O(n)**
`if fullPath == null then write error; return`	**O(1)**

findFastestPath(stations[n], Lines[m], eletric boolean)	Big-O Complexity
Total	**O(max(O(n³) + O(n²), ...))** **O(n³)**

Explanation:

1. **Initialization:** The method starts by printing station names, reading input values, and building the list of stops. These operations take **O(n)** time.
2. **Weight and Allowed Matrix Setup:** The allowed connections and weights are initialized in **O(n²)** and **O(m)** respectively.
3. **Dijkstra Calls:** The algorithm then calculates the shortest path for each segment between the stops, which involves running the `dijkstra` algorithm multiple times with a complexity of **O(n³)** for each call.
4. **Total Complexity:** The dominant term here is the repeated execution of `dijkstra`, resulting in an overall complexity of **O(n³)**.

---

3. `buildStops` Method

The `buildStops` method constructs an ordered list of stops from the source to the destination, based on the provided list of stations and intermediate stops.

`buildStops` Pseudocode Analysis

buildStops(src, dst, k integer; st[n] Stations)	Big-O Complexity
`pts[k+1]`	
`pts[1] = st[1]`	**O(1)**
for `i = 2 to k`	—
--- `idx = read`	**O(n)**
--- `pts[i] = st[idx]`	**O(n)**
`pts[k+1] = st[dst]`	**O(1)**
`return pts`	**O(1)**
Total	**O(max(O(n) + O(1), ...))** **O(n)**

Explanation:

1. **Building the Stops List:** The method iterates `k` times to construct the list of stops, where for each iteration, it reads an index and assigns a station, which takes **O(n)** per iteration.
2. **Total Complexity:** The overall time complexity is dominated by the loop that iterates `k` times, resulting in a complexity of **O(n)**.

Conclusion

The time complexity analysis of the algorithms in **US27** shows the following results:

1. The **`dijkstra`** method has a time complexity of **O(n²)**.

2. The `findFastestPath` method has a complexity of **$O(n^3)$**, primarily due to the repeated calls to `dijkstra`.
3. The `buildStops` method has a time complexity of **$O(n)$**, as it builds a list of stations in a linear fashion.

These complexities indicate that while the algorithms are manageable for small to medium-sized networks, performance may degrade for large-scale systems. Optimizations such as more efficient shortest path algorithms or better handling of large networks may be necessary for scaling up these solutions.
